

FREE EVENT

INDIA REGION

LabVIEW Expert Connect

Ask · Explore · Build Faster

📍 Online · India Standard Time (IST)



[REGISTER HERE](#)

Free entry · Limited seats



Date

3rd Sept 2026, 11 AM

Online · India Standard Time (IST)

TOPIC



Design Smart, Code Clean:
Essential LabVIEW Design Patterns

WHY ATTEND



Get Unstuck Fast



Right Tool, Right Fit



Live Q&A



Clear Next Steps

MEET YOUR HOSTS



Sintu Punnoose

Senior Applications & Systems Engineer,
Emerson NI

CLD with 11+ years of experience and deep expertise in LabVIEW, VeriStand, and advanced test system development for aerospace and industrial applications.



[Connect on LinkedIn](#)



Vijayanth Reddy Kandukuri

Team Lead, ARAV Systems

LabVIEW Champion & CLD with 17+ years of experience, Vijayanth works with LabVIEW and TestStand professionally and remains an active contributor to the LabVIEW community.



[Connect on LinkedIn](#)



Win goodies! Ask a question live to receive exclusive LabVIEW goodies.

LabVIEW Expert Connect

Sintu Punnoose

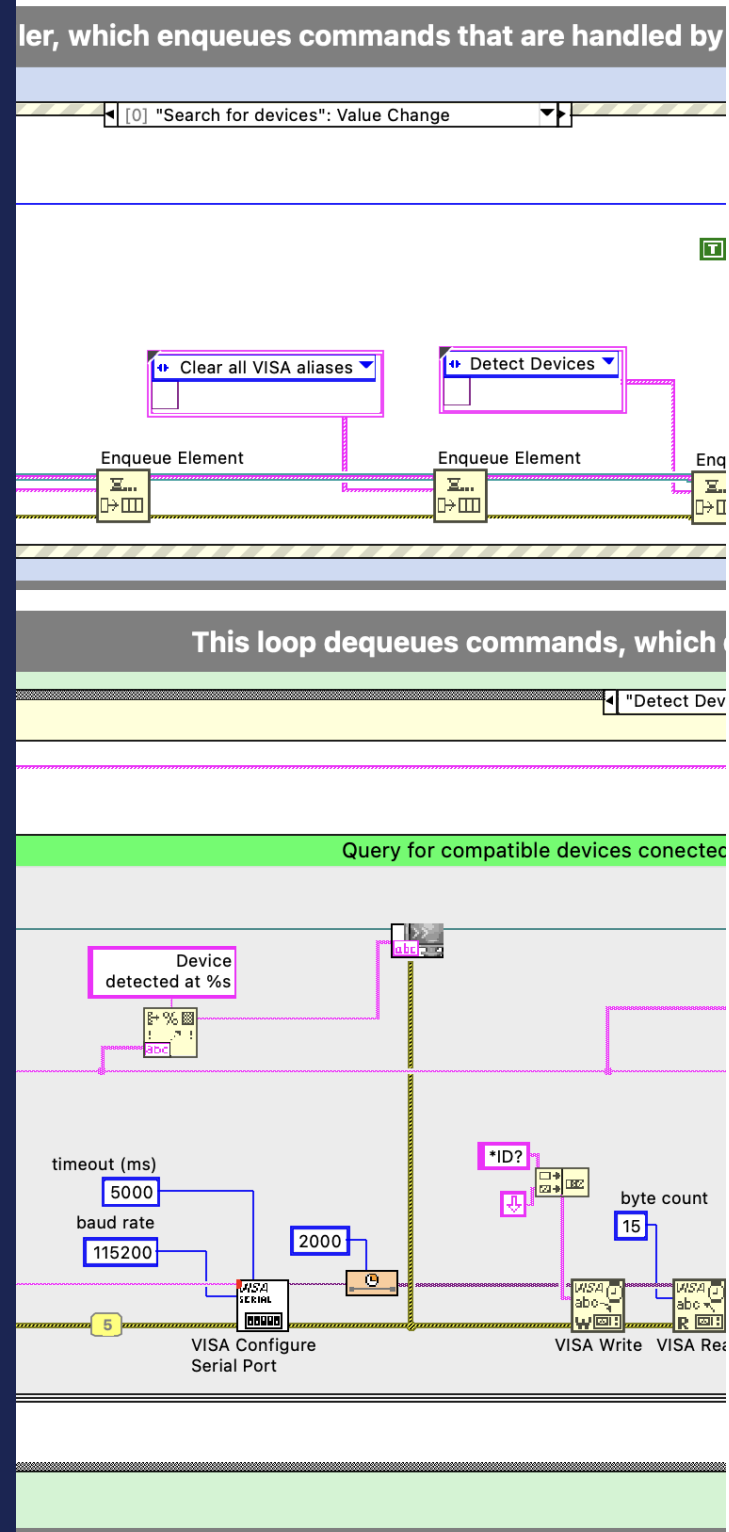
Senior Applications & Systems Engineer, NI

Vijayanth Reddy Kandukuri

Team Lead, ARAV Systems



Design Smart, Code Clean: Essential LabVIEW Design Patterns



Today's Agenda

Essential LabVIEW Design Patterns

1. What are Design Patterns & Why use them?
2. Basic Constructs:
 1. Wire as Variable
 2. Dataflow
 3. Parallelism in LV
 4. Enforcing Execution Order
3. State Machine Design Pattern
4. Event Driven State Machine
5. Producer Consumer Design Pattern

Additional Resources

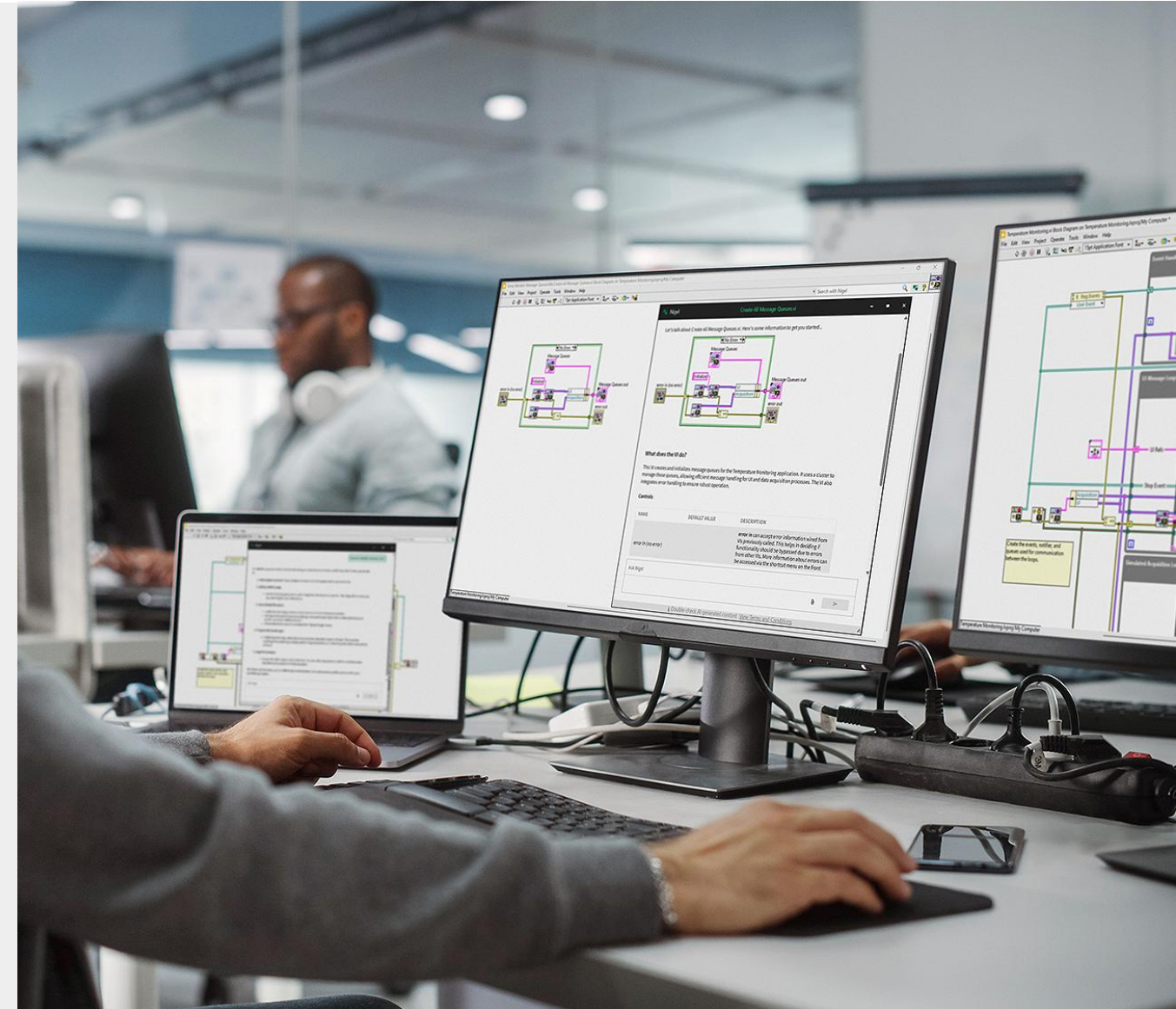
1. What are Design Patterns & Why use them?

What are Design Patterns?

A design pattern is a theoretical mechanism to **execute synchronous** or asynchronous code.

Why use them?

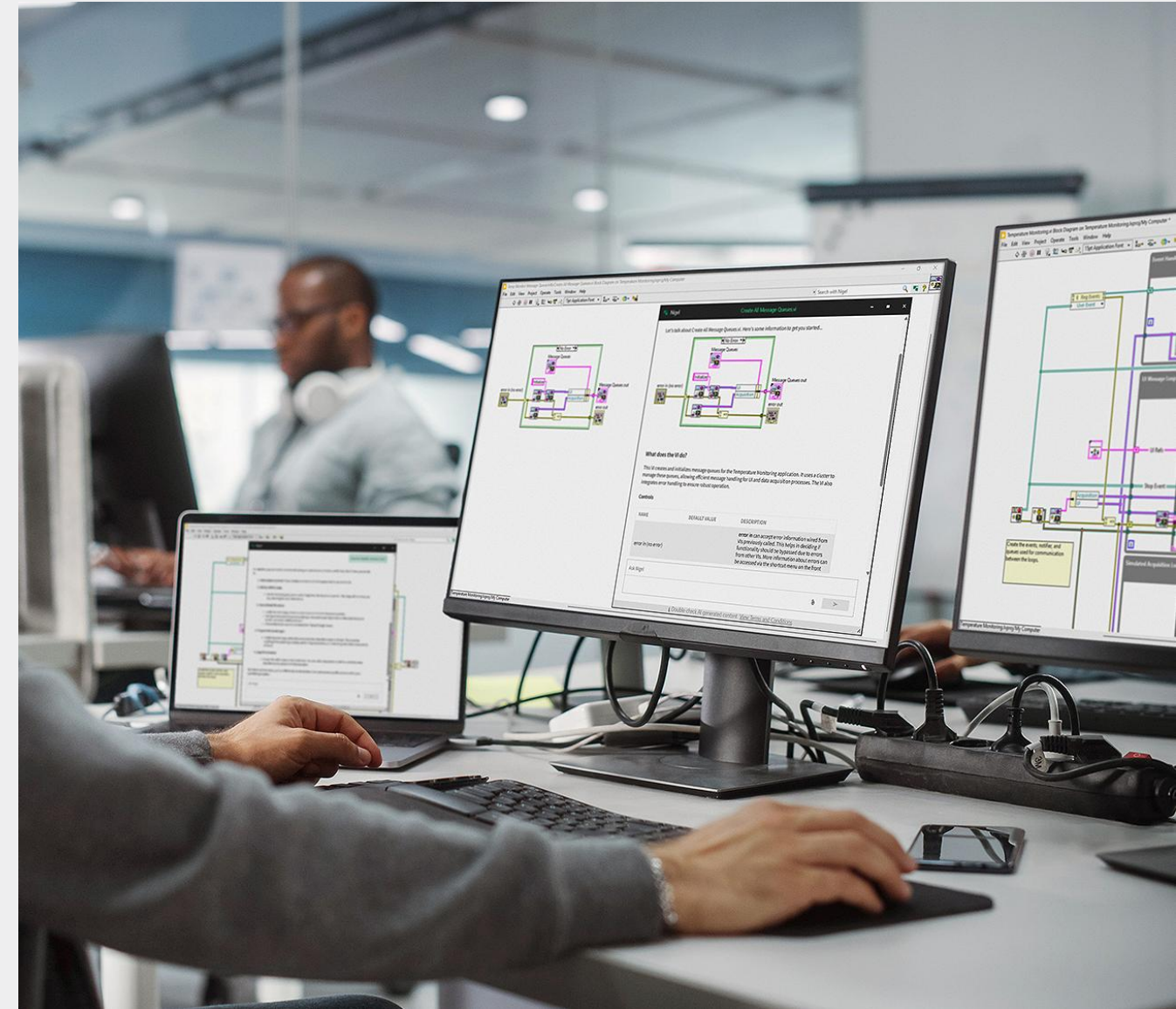
- Save time and improve the longevity of your code.
- Fine-tuned for simplicity, maintainability, and readability.
- To solve commonly occurring issues:
 - Want steps to execute in the same order?
 - Use a **sequence structure** in a loop
 - Have fixed steps whose order might change depending on external conditions?
 - Use a **state machine**
 - Want to do the above but also have a responsive UI?
 - Use the **producer consumer** design pattern



2. Basic Constructs

Understand Dataflow Before Writing More Code

- LabVIEW execution is driven by **data availability**, not by top-to-bottom reading like text-based languages.
- Using sequence structures, local variables, or artificial dependencies because they are unsure how execution order works.
- If you **understand dataflow**, half of LabVIEW becomes easier.



Demo

Wire as Variable

Dataflow

Parallelism in LV

Enforcing Execution Order

3. State Machine Design Pattern

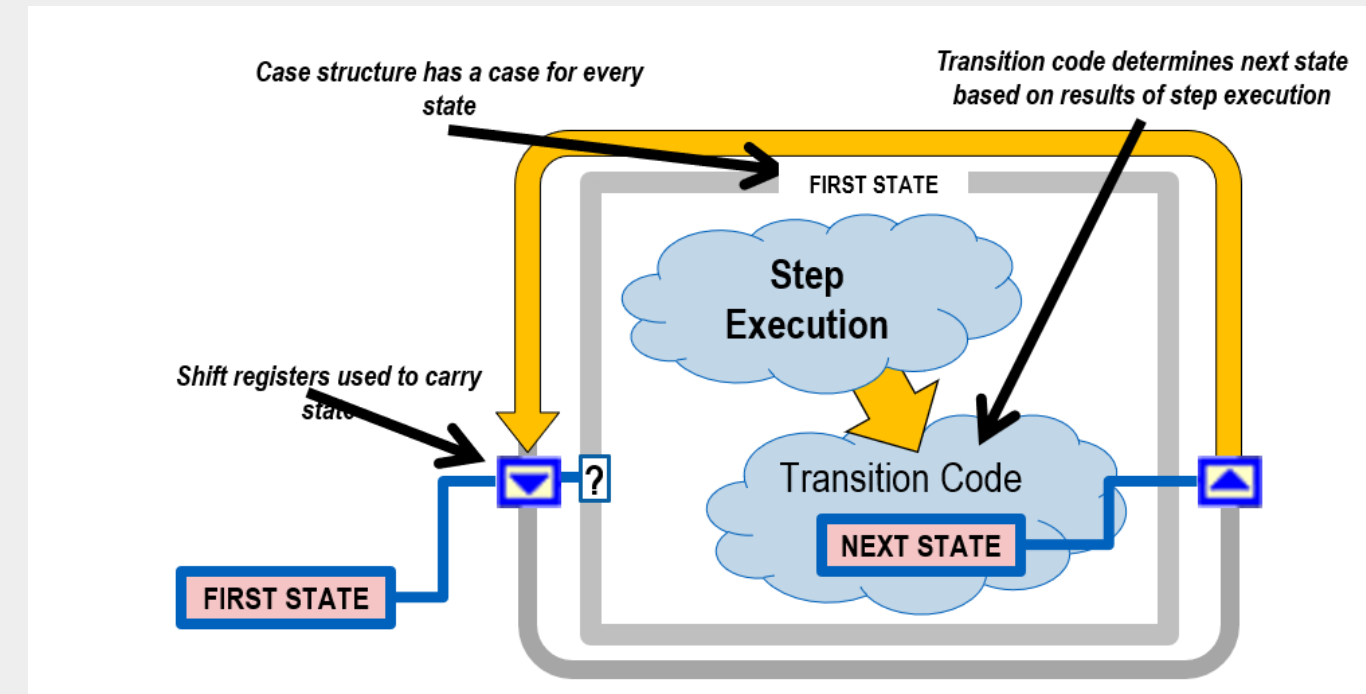
State Machine Design Pattern

State Machine: LabVIEW block diagram implementation of a **state diagram** or **flow chart**.

Given “State” is a block diagram logic to determine the next state to execute.

Anatomy of a State Machine:

1. **Typedef Enum:** containing the name of each state of the state machine
2. **Case Structure:** contains code to be executed for each state and transition code to determine the next state in the sequence
3. **While Loop:** Allows the state machine to continuously execute multiple states
4. **Shift Register:** Specifies which state will execute on the next iteration of the loop.



4. Event-Driven State Machine

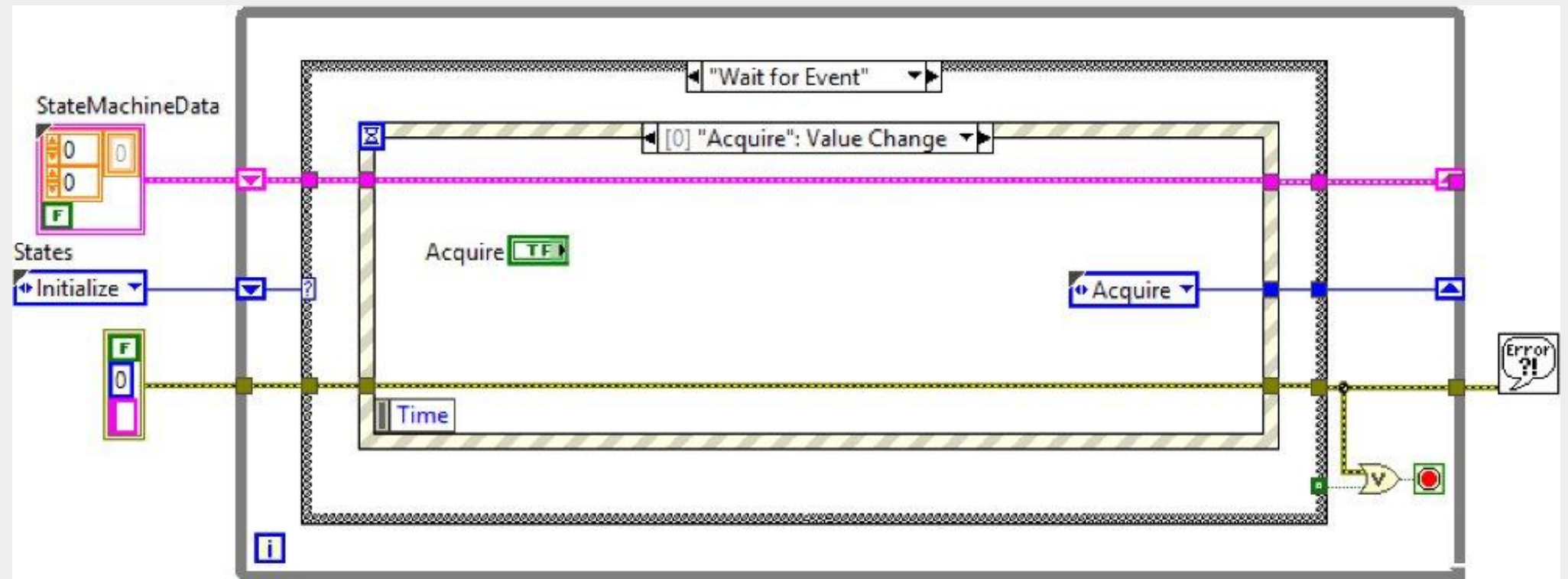
Event-Driven State Machine

The Event-Driven State Machine design pattern **combines** the **User Interface Event Handler** design pattern with the flexible transitions of the **State Machine design pattern**.

This design pattern allows you to have an **event-driven application** that can respond to events by executing a more flexible state diagram logic.

Wait for Event State:

- **Wait for an event to occur** & then **transition** to the corresponding state.
- Contains an **Event structure**, which waits for its configured events.
- When one of those **events occurs**, the Event structure passes the corresponding **next state** to the state shift register of the state machine.



Demo

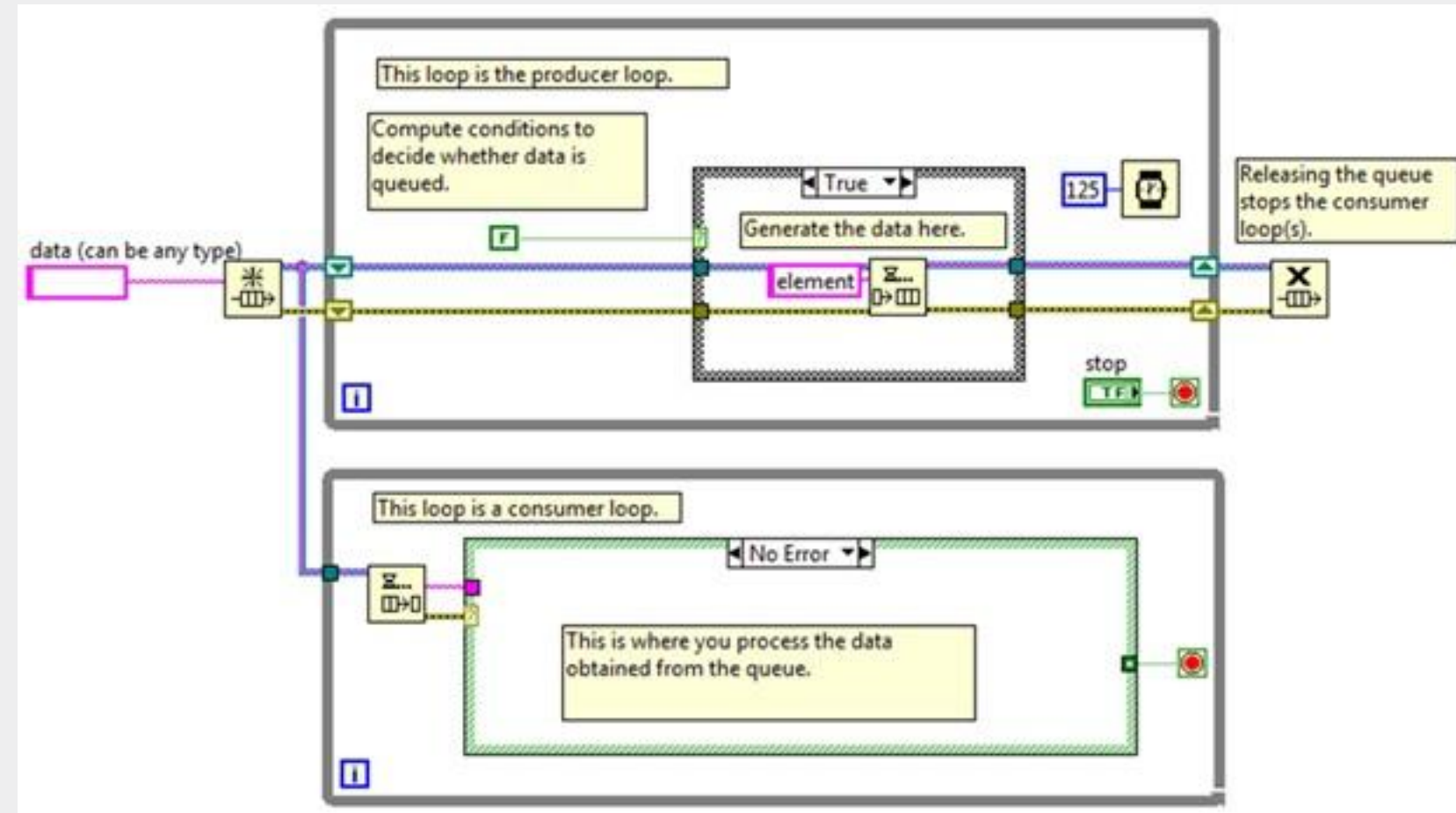
State Machine Demos

5. Producer Consumer Design Pattern

Producer Consumer Design Pattern

The **Producer/Consumer** design pattern illustrates how to use a **queue** to **share** data between multiple loops running at different rates.

- **Producer Loop:** Top loop that contains an *Enqueue Element function*.
- **Consumer Loop:** Bottom loop that contains a *Dequeue Element function*.
- **Queue:** ensures that data is not lost if the loops are running at different rates.



Demo

Producer Consumer Demo

Key Takeaways

1. **What is Design Pattern:** A design pattern is a theoretical mechanism to **execute synchronous** or asynchronous code.
2. **Why use them?**
 - Save time and improve the longevity of your code.
 - Fine-tuned for simplicity, maintainability, and readability.
3. **Basic Constructs:** Wire as Variable, Dataflow, Parallelism in LV, Enforcing Execution Order
4. **State Machine Design Pattern:** LabVIEW block diagram implementation of a flow chart.
5. **Event Driven State Machine:** combines the User Interface Event Handler design pattern with State Machine design pattern.
6. **Producer Consumer Design Pattern:** a queue to share data between multiple loops running at different rates.

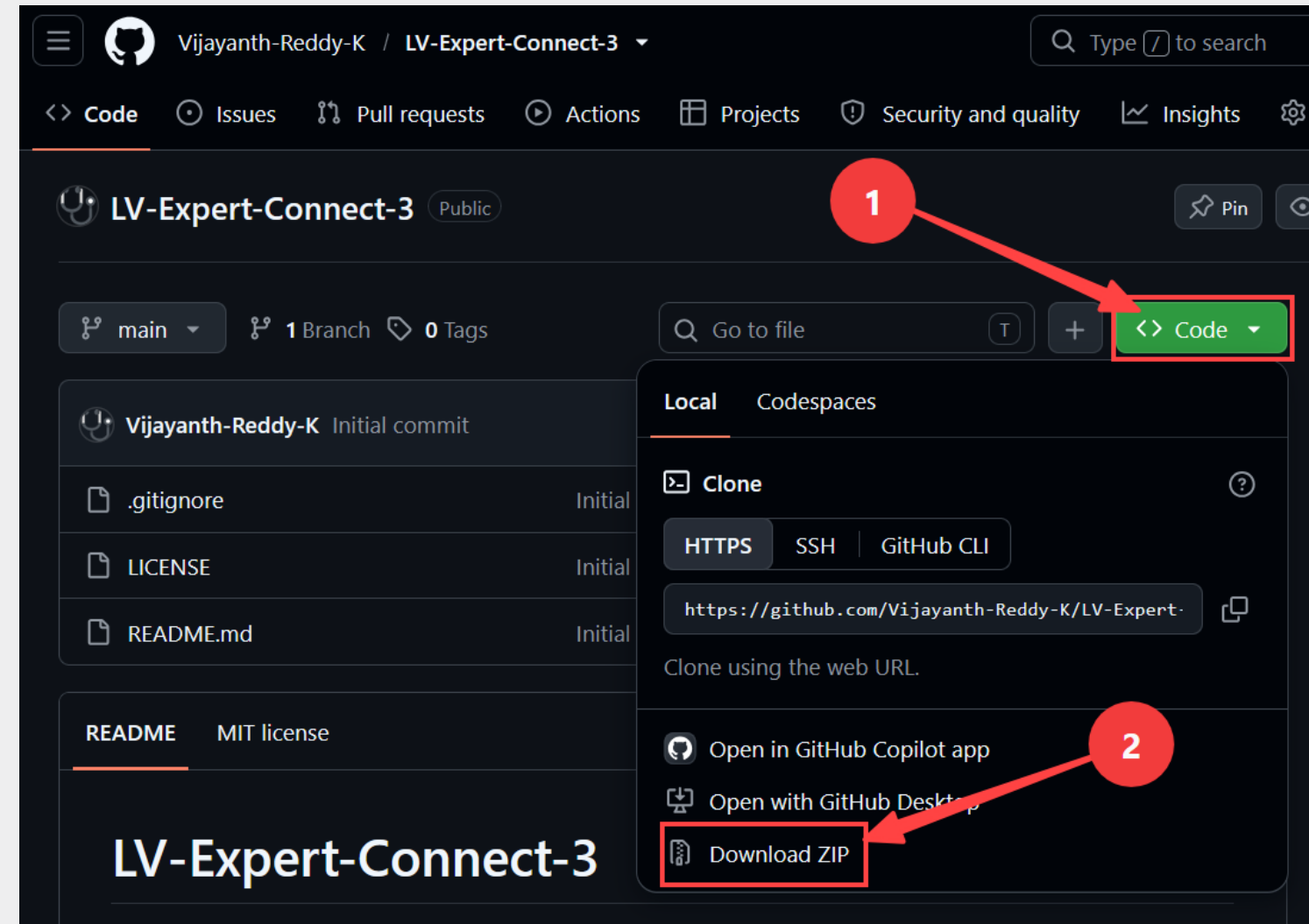
Repo Link

<https://github.com/Vijayanth-Reddy-K/LV-Expert-Connect-3>



After downloading/cloning the repo, open the project file at “Code/Design Patterns.lvproj” and look inside the Demos virtual folder.

Code will run in LabVIEW 2018 and above



References

1. [LabVIEW Tutorial](#)

- Recommended as the first link for beginners. It introduces the LabVIEW environment, programming basics, common tools, running an example, and starting a new project.

2. [NI Apps](#)

- Good for attendees who want practical exercises after the session. The guide includes hands-on exercises for building a first program, creating a practical data acquisition program, expanding acquisition to multiple devices, and using example programs as references.

3. [LabVIEW Sample Projects & Templates](#)

- Best for users ready for state machines, queued message handlers, and scalable architecture. NI states that templates and sample projects provide recommended starting points designed to support quality and scalability.

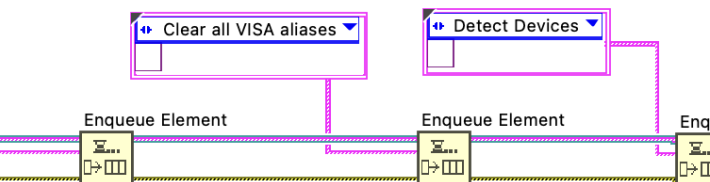
4. [Demo VIs GitHub Repo Link](#)

- Demo VI Repo shared during the LabVIEW Expert Connect Session 3. After downloading/cloning the repo, open **Design Patterns.lvproj** and reuse the Demos VIs
- Code will run in LabVIEW 2018 and above

5. [LabVIEW Community Managed Discord Servers - LabVIEW Wiki](#)

ler, which enqueues commands that are handled by

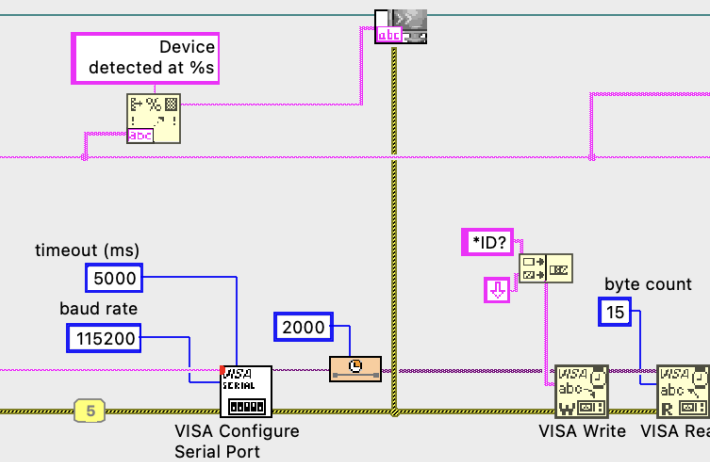
[0] "Search for devices": Value Change



This loop dequeues commands, which

"Detect Dev

Query for compatible devices connected



Q&A



NI is now part of Emerson.